    - Action: inferred from the API method (`doc.meta.read`, `task.write`)
    - Resource: derived from the API path or body (`Workspace::"custco-sales"`, `Document::"abc123"`)
    - Context: extra attributes like scopes, tenant, or optional user attribution (for example, `requested_by`)
4. **Cedar evaluates policies.** The PARC request is sent to Cedar, which evaluates the relevant policies and returns either `allow` or `deny`.
5. **Gateway enforces.** If Cedar returns `allow`, the gateway forwards the request to the API and returns the results. If Cedar returns `deny`, the gateway blocks the request with a 403.

This layering is what makes OAuth and Cedar complementary. OAuth tokens and scopes define the outer perimeter of what APIs a client can attempt to access, while Cedar policies enforce the internal rules of what the caller is actually permitted to do with a particular resource.

EXAMPLE: KATE'S BI EXPLORER (AUTHORIZATION CODE TOKEN)

When Kate connects BI Explorer to ACME, the tool acquires an access token through the Authorization Code Grant. This token includes the scopes Kate approved (`collab:workspace.read`, `collab:doc.meta.read`, and optionally `collab:event.subscribe`) and identifies Kate as the principal.

Later, BI Explorer requests a list of documents from the `custco-sales` workspace to update Kate's reporting dashboard. The GET request contains the bearer token in the header:

```
GET /workspaces/custco-sales/documents?fields=meta
Authorization: Bearer <access_token>
```

At the gateway, the token is verified and then converted into a Cedar PARC request: